



B 级达标测试实验报告

雾霾探测系统设计实验

2026 年 5 月 11 日

姓名	学号	学院	任务分工	贡献度	签名
Xxx	230xxxxxx	Xxxx 学院	xxxx	xx%	
Xxx	230xxxxxx	Xxxx 学院	xxxx	xx%	
Xxx	230xxxxxx	Xxxx 学院	xxxx	xx%	

指导教师评语:

成 绩

测试教师:

____年__月__日

实验报告内容基本要求及参考格式

- 一 问题描述
- 二 方案设计
- 三 数据获取
- 四 结果展示及分析
- 五 心得与体会

一、问题描述

近年来，雾霾天气频发，PM2.5、AQI 等关键空气质量指标直接关系到居民的身体健康与日常出行安全，不良的空气质量不仅会引发呼吸道等相关疾病，还会影响交通出行的安全性与便捷性，给人们的生产生活带来诸多不便。在此背景下，为有效解决空气质量监测不及时、数据呈现不直观等问题，实现对空气质量的实时监测、污染位置的精准定位以及天气与雾霾相关数据的可视化呈现，特设计一款基于 Web 的雾霾探测系统，为用户提供全面、精准的空气质量参考。

该系统需具备全面且实用的核心功能，首先要支持城市搜索与经纬度定位功能，能够快速响应用户的位置查询需求，并自动解析获取到的位置信息，精准定位到目标监测区域，为后续数据获取与展示奠定基础。同时，系统需对接第三方天气 API，通过稳定的数据接口，精准获取实时天气状况、空气质量等级以及各类污染物浓度等核心数据，确保数据来源的可靠性与时效性。

在数据呈现与服务保障方面，系统需在前端页面动态展示温度、湿度、AQI 指数、PM2.5 浓度、能见度等关键指标，让用户能够直观了解当前空气质量状况；此外，还需呈现近 6 小时内温度与湿度的变化趋势折线图，助力用户清晰掌握环境参数的动态变化规律。后端服务则需保证运行稳定，支持跨域访问，确保不同环境下的正常使用，同时提升数据解析的准确性，全方位满足用户对雾霾及空气质量监测的相关需求。

二、方案设计

2.1 系统架构

前端层：HTML5 + 静态页面，适配手机与PC端，负责数据展示与交互；
后端层：Node.js + Express，提供RESTful API，处理定位、数据请求与整合；
数据层：和风天气 API，提供实时天气、空气质量、24小时预报数据。

2.2 技术选型

模块	技术/工具	用途
后端框架	Express	搭建Web服务器，路由管理

模块	技术/工具	用途
网络请求	<i>Axios</i>	调用第三方 <i>API</i> ，并发请求
跨域处理	<i>CORS</i>	允许前端跨域访问接口
数据源	和风天气 <i>API</i>	天气、 <i>AQI</i> 、 <i>PM2.5</i> 、预报数据
前端	<i>HTML5</i> +原生 <i>JS</i>	数据展示、趋势图绘制
运行环境	<i>Node.js</i>	后端服务运行

2.3 核心流程

当用户在前端界面输入城市名称或经纬度信息后，前端页面会自动发起请求，调用后端的/api/weather 接口传递用户输入的位置参数；后端接收到请求后，立即调用地理编码或逆地理编码 API，对用户输入的非标准位置信息进行解析处理，将其转换为统一标准的经纬度坐标，确保位置定位的精准性；随后，后端通过并发请求的方式，同步调用实时天气、空气质量以及 24 小时预报等相关接口，高效获取多维度的环境数据，避免因串行请求导致的响应延迟；获取所有原始数据后，后端会对数据进行清洗过滤，剔除无效、异常数据，同时根据相关算法计算出能见度指标，并对温度、湿度等数据进行整理组装，生成符合前端展示需求的趋势数据，完成数据处理后将所有整理好的数据统一返回给前端；最后，前端接收后端返回的数据，按照预设的界面布局完成各项指标的动态渲染展示，并根据趋势数据绘制出温度与湿度的折线图，让用户能够直观、清晰地查看当前及近期的环境变化情况。

2.4 功能模块设计

定位解析模块作为流程起点，全面支持用户输入城市名搜索、直接输入经纬度定位两种方式，同时集成逆地理编码功能，能够对用户输入的非标准位置信息进行精准解析，将城市名转换为标准经纬度坐标，也可通过经纬度反查对应地理信息，，为后续数据获取提供可靠的位置依据；数据获取模块承接定位解析结果，通过并发请求的方式同步调用第三方接口，高效获取天气实况、空气质量、小时

预报等多维度环境数据,相较于串行请求大幅提升数据获取效率,避免响应延迟,同时保障数据来源的可靠性与时效性;数据计算模块对获取的原始数据进行深度处理,结合实时天气、空气湿度、PM2.5 浓度等核心参数,通过预设算法精准计算出能见度指标,填补原始数据中能见度参数的空白,为用户提供更全面的环境参考;数据封装模块则对处理后的所有数据进行规范化整理,统一响应格式,将温度、湿度、AQI、PM2.5、能见度及趋势数据等封装成前端可直接调用的结构,降低前端渲染难度;异常处理模块全程保驾护航,对流程中可能出现的网络连接错误、第三方 API 权限不足、用户输入参数缺失等各类异常情况进行统一捕获与处理,避免系统崩溃,同时返回清晰的异常提示,保障系统运行的稳定性与用户体验,各模块依次推进,最终由前端接收封装后的数据,完成指标渲染与趋势折线图绘制,实现系统核心功能。

三、数据获取

3.1 数据源

使用和风天气付费/免费接口,获取以下数据:1)实时天气:温度、体感温度、天气现象、风向风力、湿度;2)空气质量: *AQI* 指数、等级、*PM2.5*、*PM10*、*NO₂* 浓度;3)24 小时预报:近 6 小时温度、湿度,用于趋势图;4)地理编码:城市名与经纬度互转。

3.2 数据获取方式

数据获取方式上:1)地理编码: */geo/v2/city/lookup*,将城市名转为经纬度;2)逆地理编码:同接口,经纬度解析为城市名称;3)实时天气: */v7/weather/now*,获取当前天气;4)空气质量: */airquality/v1/current*,获取 *AQI* 与污染物;5)小时预报: */v7/weather/24h*,获取近 24 小时数据。

3.3 关键数据处理

对于数据的处理,我们采用如下办法:1)污染物解析:从 *pollutants* 数组中提取 *pm2p5*、*pm10*、*no2* 的 *concentration.value*;2) *AQI* 匹配:优先选取 *cn - mee/cn - mep* 国标 *AQI*;3)能见度计算:根据雾霾/雾天气、*PM2.5*、湿度分级计算;4)趋势数据:截取前 6 小时数据,格式化时间标签。

四、结果展示及分析

整体界面展示：



4.1 功能实现结果

1. 定位功能



支持 `/api/weather? city = 北京` 按城市查询；

支持 `/api/weather? lon = 116.23&lat = 39.54` 按坐标查询；

自动解析为 “北京市 北京 北京” 标准名称。

2. 数据展示



实时显示：温度、体感温度、天气状况、湿度、风力、能见度；

空气质量：AQI 数值、等级、PM2.5/PM10/NO2 浓度；

近 6 小时温度、湿度折线图，趋势清晰。

3. 接口稳定性

多 URL 重试机制，地理编码接口容错率高；

代码

```
const urls = [
  `${BASE}/geo/v2/city/lookup?location=...`,
  `${BASE}/v2/city/lookup?location=...`,
  `${BASE}/city/lookup?location=...`,
];
for (const url of urls) {
  try {
    const res = await axios.get(url);
    // 成功就返回
  } catch (e) {
    // 失败就继续试下一个 URL
    continue;
  }
}
```

后端对地理编码与逆地理编码接口提供 3 个兼容路径，一个接口失败自动尝试下一个，避免因接口版本或路径问题导致定位失败，定位解析更稳定。

并发请求提升响应速度：

代码

```
const [weatherRes, airRes, forecastRes] = await Promise.all([
  axios.get(天气接口),
  axios.get(空气质量接口),
  axios.get(预报接口),
]);
```

使用 *Promise.all* 同时请求天气、空气质量、24 小时预报三个接口，采用并发方式获取数据，减少等待时间，接口响应更快。

完善异常捕获，返回友好错误提示。

代码

```
try {
  // 业务逻辑
} catch (error) {
  if (error.response?.status === 403) {
    return res.json({ status: 'error', message: 'API Key 权限不足' });
  } else if (error.response?.status === 404) {
    return res.json({ status: 'error', message: 'API 端点不存在' });
  } else {
    return res.json({ status: 'error', message: error.message });
  }
}
```

后端使用 *try/catch* 全局捕获异常，对参数缺失、API 权限不足、接口不存在、网络错误等情况统一处理，并返回明确中文提示，前端可直接展示，系统运行更健壮。

4.2 结果分析



AQI、PM2.5 等数据与和风天气官方一致；支持城市搜索与定位，满足日常监测需求。

五 心得与体会

通过本次雾霾探测系统设计实验，我们完整经理了前后端分离开发、API 调用、数据解析与可视化、服务端异常处理的全流程开发。实验过程中，围绕系统核心需求，从需求分析到方案设计，再到代码实现与测试调优，每一个环节都亲力亲为，不仅巩固了已学的 Web 开发基础，更对前后端协同工作的逻辑的有了更深刻的理解，提升了自身的综合开发素养。

技术上，我们学会搭建稳定高效的后端服务，深入掌握了并发请求技巧与异常处理方法，能够灵活应对多接口同步调用的场景，同时清晰理解了地理编码的实现原理与空气质量相关数据的结构特点，能够精准解析复杂的嵌套数据。工程能力上，逐步学会了需求分析、方案设计、代码实现、测试调优的完整开发流程，培养了规范化的开发思维，学会了从整体出发规划系统架构，兼顾功能实现与用户体验。此外，我们还成功解决了实验过程中遇到的 API 路径兼容、污染物数据嵌套解析、跨域访问、参数校验等多个实际问题，积累了宝贵的问题解决经验，提升了自身的故障排查与调试能力。

该系统可直接应用于日常空气质量监测，为居民出行、健康防护提供精准的环境参考，切实解决了雾霾天气下人们对空气质量信息的需求，具备较强的实际使用价值。同时，实验过程中积累的前后端分离开发经验、第三方 API 对接技巧、数据处理与可视化方法，以及异常处理的思路，都为后续开展物联网环境监测等相关项目提供了可借鉴的实践方案。